

# Clay Embeddings Tutorial

Andrés Padrón Quintana

r Sys.Date()

## Contents

|                                           |          |
|-------------------------------------------|----------|
| <b>Introduction</b>                       | <b>2</b> |
| <b>Background</b>                         | <b>2</b> |
| What is an Earth Embedding? . . . . .     | 2        |
| What is Clay? . . . . .                   | 3        |
| <b>Study Area</b>                         | <b>3</b> |
| <b>Part 1: Generating Clay Embeddings</b> | <b>4</b> |
| Loading Clay . . . . .                    | 4        |
| Extracting Embeddings . . . . .           | 4        |
| Output . . . . .                          | 4        |
| <b>Part 2: Google Earth Engine</b>        | <b>5</b> |
| Objective . . . . .                       | 5        |
| Creating City Locations . . . . .         | 5        |
| Loading Sentinel-2 . . . . .              | 5        |
| <b>Sentinel-2 Spectral Bands</b>          | <b>6</b> |
| <b>Environmental Indices</b>              | <b>6</b> |
| NDVI . . . . .                            | 6        |
| NDWI . . . . .                            | 7        |
| NDBI . . . . .                            | 7        |
| <b>Extracting Statistics</b>              | <b>7</b> |
| <b>Exporting Results</b>                  | <b>8</b> |

|                                               |           |
|-----------------------------------------------|-----------|
| <b>Part 3: Merging Clay and Sentinel Data</b> | <b>8</b>  |
| Loading Data . . . . .                        | 8         |
| Dataset Dimensions . . . . .                  | 9         |
| Merge . . . . .                               | 9         |
| <b>Part 4: Principal Component Analysis</b>   | <b>9</b>  |
| Why PCA? . . . . .                            | 9         |
| Selecting Embedding Dimensions . . . . .      | 9         |
| Running PCA . . . . .                         | 10        |
| Visualization . . . . .                       | 10        |
| <b>Results</b>                                | <b>11</b> |
| Environmental Indicators . . . . .            | 11        |
| Observations . . . . .                        | 11        |
| <b>Conclusions</b>                            | <b>11</b> |

## Introduction

This tutorial presents a complete workflow for generating geospatial embeddings using the **Clay Foundation Model** and comparing them with environmental indicators derived from **Sentinel-2 satellite imagery** through **Google Earth Engine (GEE)**.

The objective is to understand how foundation models for Earth Observation can compress large volumes of satellite information into dense vector representations (embeddings) and explore whether these embeddings capture meaningful environmental characteristics such as vegetation, moisture, and urbanization.

The workflow consists of four stages:

1. Generate embeddings using Clay.
2. Extract Sentinel-2 spectral information using Google Earth Engine.
3. Compute environmental indices (NDVI, NDWI, and NDBI).
4. Compare both representations using Principal Component Analysis (PCA).

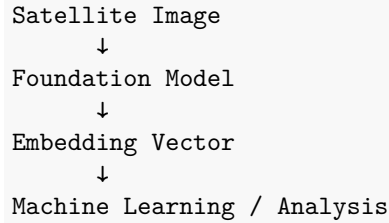
## Background

### What is an Earth Embedding?

An Earth embedding is a numerical vector representation of a geographic location learned from satellite imagery.

Instead of working directly with raw pixels, a foundation model transforms an image into a compact representation that captures relevant spatial, spectral, and environmental information.

Conceptually:



These embeddings can then be used for:

- Land cover classification
- Environmental monitoring
- Biodiversity analysis
- Climate studies
- Urbanization detection
- Downstream machine learning tasks

## What is Clay?

Clay is an open-source geospatial foundation model designed for Earth Observation applications.

It processes multispectral satellite imagery and generates dense vector embeddings that summarize information contained in the original image.

For this tutorial, Clay produces embeddings with:

**1024 dimensions**

Each dimension is a learned feature that may encode information about:

- Vegetation
- Water presence
- Soil characteristics
- Urban structures
- Spectral patterns
- Geographic context

---

## Study Area

Five major Mexican cities were selected:

| City        | Latitude | Longitude |
|-------------|----------|-----------|
| Mexico City | 19.4326  | -99.1332  |
| Guadalajara | 20.6597  | -103.3496 |
| Monterrey   | 25.6866  | -100.3161 |
| Mérida      | 20.9674  | -89.5926  |
| Oaxaca      | 17.0732  | -96.7216  |

These cities were chosen because they exhibit different environmental and urban characteristics.

# Part 1: Generating Clay Embeddings

## Loading Clay

The Clay model is loaded locally through Python.

```
from clay import ClayModel  
  
model = ClayModel.from_pretrained(...)
```

## Extracting Embeddings

For each city:

1. Download Sentinel imagery.
2. Preprocess spectral bands.
3. Pass the image through Clay.
4. Extract the embedding vector.

Example:

```
embedding = model.encode(image)
```

## Output

Each city produces a vector of:

```
1024 dimensions
```

Example:

```
city  
lat  
lon  
dim_0  
dim_1  
...  
dim_1023
```

These embeddings were exported into:

```
clay_mexico_embeddings.csv
```

Dataset dimensions:

```
5 rows × 1027 columns
```

where:

- 3 metadata columns
- 1024 embedding dimensions

## Part 2: Google Earth Engine

### Objective

The goal is to obtain environmental indicators from Sentinel-2 imagery that can be compared against Clay embeddings.

### Creating City Locations

The cities are defined as geographic points.

```
var cities = ee.FeatureCollection([
  ee.Feature(
    ee.Geometry.Point([-99.1332, 19.4326]),
    {city: 'CDMX'}
  ),
  ee.Feature(
    ee.Geometry.Point([-103.3496, 20.6597]),
    {city: 'Guadalajara'}
  ),
  ee.Feature(
    ee.Geometry.Point([-100.3161, 25.6866]),
    {city: 'Monterrey'}
  ),
  ee.Feature(
    ee.Geometry.Point([-89.5926, 20.9674]),
    {city: 'Merida'}
  ),
  ee.Feature(
    ee.Geometry.Point([-96.7216, 17.0732]),
    {city: 'Oaxaca'}
  )
]);
```

### Loading Sentinel-2

We use the harmonized Sentinel-2 Surface Reflectance dataset.

```
var image = ee.ImageCollection(
  "COPERNICUS/S2_SR_HARMONIZED"
)
.filterBounds(cities)
.filterDate(
  '2025-01-01',
  '2025-12-31'
)
.filter(
```

```
ee.Filter.lt(
  'CLOUDY_PIXEL_PERCENTAGE',
  20
)
)
.median();
```

## Sentinel-2 Spectral Bands

The following bands were selected:

| Band | Description         |
|------|---------------------|
| B2   | Blue                |
| B3   | Green               |
| B4   | Red                 |
| B8   | Near Infrared (NIR) |
| B11  | SWIR 1              |
| B12  | SWIR 2              |

These bands are commonly used for environmental monitoring.

## Environmental Indices

### NDVI

Normalized Difference Vegetation Index

Measures vegetation density and health.

Formula:

$$NDVI = \frac{NIR - RED}{NIR + RED}$$

Implemented in GEE:

```
var ndvi = image
  .normalizedDifference(
    ['B8', 'B4']
  )
  .rename('NDVI');
```

Interpretation:

| Value     | Meaning           |
|-----------|-------------------|
| < 0       | Water             |
| 0 - 0.2   | Bare soil         |
| 0.2 - 0.5 | Sparse vegetation |

| Value | Meaning          |
|-------|------------------|
| > 0.5 | Dense vegetation |

## NDWI

Normalized Difference Water Index

Measures water content and surface moisture.

Formula:

$$NDWI = \frac{GREEN - NIR}{GREEN + NIR}$$

Implemented as:

```
var ndwi = image
  .normalizedDifference(
    ['B3', 'B8']
  )
  .rename('NDWI');
```

## NDBI

Normalized Difference Built-up Index

Measures urbanized and built-up areas.

Formula:

$$NDBI = \frac{SWIR - NIR}{SWIR + NIR}$$

Implemented as:

```
var ndbi = image
  .normalizedDifference(
    ['B11', 'B8']
  )
  .rename('NDBI');
```

## Extracting Statistics

A 10 km buffer was created around each city.

```
var cityBuffers = cities.map(
  function(feature) {
    return feature.buffer(10000);
  }
);
```

Mean values were extracted using:

```
var cityStats =  
features.reduceRegions({  
  
  collection: cityBuffers,  
  
  reducer:  
    ee.Reducer.mean(),  
  
  scale: 10  
  
});
```

## Exporting Results

Results were exported to Google Drive.

```
Export.table.toDrive({  
  
  collection: cityStats,  
  
  description:  
    'mexico_cities_sentinel_indices_buffer10km',  
  
  fileFormat: 'CSV'  
  
});
```

Output:

```
mexico_cities_sentinel_indices_buffer10km.csv
```

## Part 3: Merging Clay and Sentinel Data

### Loading Data

```
import pandas as pd  
  
clay = pd.read_csv(  
    "clay_mexico_embeddings.csv"  
)  
  
sentinel = pd.read_csv(  
    "mexico_cities_sentinel_indices_buffer10km.csv"  
)
```



## Dataset Dimensions

Clay:

```
5 × 1027
```

Sentinel:

```
5 × 12
```

## Merge

```
sentinel["city"] = (
    sentinel["city"]
    .str.lower()
)

data = clay.merge(
    sentinel,
    on="city",
    how="inner"
)
```

Result:

```
5 × 1038
```

## Part 4: Principal Component Analysis

### Why PCA?

The Clay embeddings contain:

```
1024 dimensions
```

which cannot be visualized directly.

PCA reduces the dimensionality while preserving as much variance as possible.

### Selecting Embedding Dimensions

```
embedding_cols = [
    col for col in data.columns
    if col.startswith("dim_")
]

X = data[embedding_cols]
```

## Running PCA

```
from sklearn.decomposition import PCA

pca = PCA(
    n_components=2
)

pcs = pca.fit_transform(X)

data["PC1"] = pcs[:, 0]
data["PC2"] = pcs[:, 1]
```

Explained variance:

```
PC1: 60.10%
PC2: 34.78%
```

Total:

```
94.88%
```

## Visualization

```
import matplotlib.pyplot as plt

plt.figure(figsize=(8,6))

plt.scatter(
    data["PC1"],
    data["PC2"]
)

for _, row in data.iterrows():
    plt.annotate(
        row["city"],
        (
            row["PC1"],
            row["PC2"]
        )
    )

plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title(
    "Clay Embeddings PCA: Mexican Cities"
)

plt.show()
```

# Results

## Environmental Indicators

| City        | NDVI  | NDWI   | NDBI   |
|-------------|-------|--------|--------|
| CDMX        | 0.175 | -0.252 | 0.026  |
| Guadalajara | 0.185 | -0.264 | 0.041  |
| Monterrey   | 0.265 | -0.284 | -0.022 |
| Mérida      | 0.380 | -0.387 | 0.017  |
| Oaxaca      | 0.352 | -0.421 | 0.074  |

## Observations

The PCA reveals meaningful spatial organization within the embedding space.

Cities with higher vegetation levels, such as Mérida and Oaxaca, appear separated from highly urbanized cities such as Mexico City and Guadalajara.

Monterrey appears relatively isolated, likely reflecting its unique semi-arid environment and distinct land cover patterns.

These findings suggest that Clay embeddings encode environmental information that is consistent with spectral indices derived directly from Sentinel-2 imagery.

## Conclusions

This tutorial demonstrated a complete Earth Observation workflow combining:

- Clay Foundation Model
- Sentinel-2 imagery
- Google Earth Engine
- Environmental indices
- Principal Component Analysis

The experiment provides preliminary evidence that Clay embeddings capture meaningful geographic and environmental characteristics.

Although only five cities were analyzed, the workflow establishes a foundation for larger-scale studies involving:

- Hundreds of locations
- Additional foundation models
- Downstream machine learning tasks
- Comparison against AlphaEarth Foundations

Future work will extend this analysis to other geospatial foundation models such as Prithvi, SatCLIP, GeoCLIP, and AlphaEarth.